

“초등부 2번 / 중등부 1번. 두 배” 문제 풀이

작성자: 김준원

부분문제 1, 2

가장 쉽게 생각할 수 있는 방법은 “앞에서부터 차례대로 2를 곱해나가기”일 것이다. 수열의 두 번째 원소부터 마지막 원소까지 차례대로 보면서, 직전의 원소보다 크거나 같게 될 때까지 2를 곱해나가는 것이다. 일종의 그리디 알고리즘이다.

부분문제 1, 2에서는 수열의 모든 원소가 2의 거듭제곱이라는 조건이 존재하므로, 각 원소에 연산을 (부분문제 1) 1번, (부분문제 2) 18번 이하 적용하면 오름차순으로 만들 수 있다. 따라서, A_i 의 최대 크기를 상수로 볼 때 $O(N)$ 의 시간복잡도에 문제를 해결할 수 있다.

부분문제 3

부분문제 1, 2와 같은 시간 복잡도로 문제를 해결할 수 있다. 단, 모든 원소가 2의 거듭제곱이라는 조건이 존재하지 않기 때문에, 두 배 연산을 취한 결과 각 원소가 직전 원소의 2배 미만까지 커질 수 있다. 따라서 i 번째 원소에 최대 $18 + i$ 번까지 연산을 적용하게 될 수 있으며, 시간 복잡도는 $O(N^2)$ 이다.

부분문제 4

부분문제 1, 2, 3의 풀이는 시간 복잡도도 느리고, 수들이 너무 커질 수 있기 때문에 부분문제 4, 5, 6에는 그대로 적용할 수 없다. 따라서, 다른 관찰이 필요하다.

먼저, 부분문제 4에서는 각 원소가 2 또는 3이므로, 경우를 나누어 생각할 수 있게 된다. 인접한 두 원소 A_{i-1} 와 A_i 를 고려해보자. A_i 에 연산을 적용할 필요가 있는 경우는 “ $A_{i-1} = 3, A_i = 2$ 일 때” 뿐이다. A_{i-1}, A_i 가 이와 다른 값을 가질 때에는 $A_{i-1} \leq A_i$ 이므로 따로 연산을 적용할 필요가 없다.

그리고, 위 경우 A_i 에 연산을 적용한 뒤에는 $A_i = 4$ 가 되므로, A_i 이후의 모든 원소 ($A_{i+1}, A_{i+2}, \dots$)보다 A_i 의 값이 커지게 된다. 따라서, A_i 이후의 모든 원소를 2배로 만들어줄 필요가 있다. 그리고 이후의 모든 원소가 **똑같이** 2배가 된 후에는, 이후의 원소들에 대해서도 위와 동일한 논리를 적용할 수 있게 됨을 알 수 있다. 따라서, 이를 $i = 2, 3, \dots, N$ 에 대해 순서대로 적용하면 문제를 풀 수 있다. 시간 복잡도는 $O(N)$ 이다.

부분문제 5

부분문제 5에서도 배열이 초기에 내림차순이므로 부분문제 4와 비슷한 관찰을 할 수 있다.

부분문제 6

앞선 부분문제들의 관찰에서 인접한 두 원소 A_{i-1} 과 A_i 의 관계를 사용했듯, 전체 문제를 풀 때에도 이 관계를 사용한다. 원소의 실제 값이 무엇인지보다, 이전 원소보다 크거나 같아지기 위해 연산을 몇 번 사용하는지가 중요하다.

M_i 를 $A_{i-1} \leq A_i \times 2^x$ 인 가장 작은 정수 x 의 값이라고 두자. 이제, 인덱스 i 에 대해 “두 배” 연산을 적용하는 것은 M_i 를 1 감소시키고 M_{i+1} 를 1 증가시키는 것이며, 목표는 $M_1, \dots, M_N$ 을 모두 양수로 만드는 것이다. 따라서, 배열 M 의 두 번째 원소부터 순회하면서, 양수 값을 가지고 있으면 필요한 횟수만큼 연산을 적용하면 된다. 시간 복잡도는 $O(N)$ 이다.

“중등부 2번. 회의 장소” 문제 풀이

작성자: 박선재

부분문제 1

각 질의마다 방문한 가능한 순서를 모두 시도해 볼 수 있다. 가능한 순서들에 대해 회의 세트의 피로도를 전부 계산하고, 그중 최솟값을 출력하면 된다.

부분문제 2

각 질의마다 동적 계획법을 사용해 정답을 구할 수 있다. 다음과 같이 **best**라는 배열을 정의한다.

$\text{best}[S][i]$: 부분집합 S 에 해당하는 집들만을 방문하고, 그 중 i 번째 집을 마지막으로 방문하는 모든 방법 가운데, 그때까지의 피로도 중 최솟값

구현 시에는 S 를 집합으로 표현하기보다는, 비트 표현 등을 사용할 수 있을 것이다.

위와 같은 정의는 동적 계획법으로 외판원 문제(traveling salesman problem)를 해결할 때의 정의와 유사하며, **best** 배열을 $O(N^2 2^N)$ 의 복잡도로 계산할 수 있다. 각 질의에 대해 **best** 배열을 독립적으로 계산하여 문제를 해결할 수 있으므로, 전체 시간복잡도는 $O(QN^2 2^N)$ 이다.

부분문제 3

한 가지 관찰이 필요하다. 각 질의에 대해서, 주어진 순서에 대한 회의 세트의 피로도는 수직선에서 각 회의 비용에 대응되는 모든 점을 한 번씩 방문할 때 필요한 이동 거리와 같다. 따라서, 이때의 최소 이동 거리가 최소 피로도이므로, 최대 회의 비용과 최소 회의 비용의 차가 최소 피로도가 된다. 각 질의에 대해 $O(N^2)$ 의 복잡도로 최대 회의 비용과 최소 회의 비용을 구할 수 있다. 따라서 전체 시간복잡도 $O(N^2 Q)$ 에 문제를 해결할 수 있다.

부분문제 4

회의 세트에 참가하는 집들의 좌표를 작은 것부터 순서대로 $a_1, a_2, \dots, a_k$ 라고 했을 때, a_i 에서 모일 때 회의 비용을 b_i 라고 하자. 이때, $1 \leq i < k$ 에 대해 $b_{i+1} - b_i$ 를 생각하면, $(2i - k) \cdot (a_{i+1} - a_i)$ 이다. 따라서, $2i < k$ 이면 $b_{i+1} - b_i < 0$ 이고, $2i > k$ 이면 $b_{i+1} - b_i > 0$ 이다. 그러므로, a_1 또는 a_k 에서 모일 때 회의 비용이 최대가 되고, $a_{\lfloor (k+1)/2 \rfloor}$ 에서 모일 때 회의 비용이 최소가 된다. 따라서, $O(N)$ 의 복잡도로 각 회의 세트의 최소 피로도를 계산할 수 있고, 전체 시간복잡도 $O(QN)$ 에 문제를 해결할 수 있다.

부분문제 5

앞선 부분문제들의 관찰에서, a_1 에 있는 집에서 모일 경우 회의 비용을 계산하면, $(a_1 + a_2 + \dots + a_k) - k \cdot a_1$ 이다. 비슷하게, a_k 에 있는 집에서 모일 경우 회의 비용은 $k \cdot a_k - (a_1 + a_2 + \dots + a_k)$ 이다. 마지막으로, $a_{\lfloor (k+1)/2 \rfloor}$ 에 있는 집에 모일 경우, 회의 비용은 $(\lfloor (k+1)/2 \rfloor \cdot a_{\lfloor (k+1)/2 \rfloor}) - (a_1 + a_2 + \dots + a_{\lfloor (k+1)/2 \rfloor}) + (a_{\lfloor (k+1)/2 \rfloor + 1} + a_{\lfloor (k+1)/2 \rfloor + 2} + \dots + a_k) - \lfloor k/2 \rfloor \cdot a_{\lfloor (k+1)/2 \rfloor}$ 이다.

따라서, 회의 세트에 참여하는 집들에 대해, 좌표의 최솟값, 좌표의 최댓값, 좌표의 중간값, 좌표의 합을 계산할 수 있으면 문제를 해결할 수 있다.

집들의 좌표가 오름차순으로 정렬되어 있기 때문에, 이진 탐색을 통해 집의 좌표가 l 이상 r 이하인 집의 번호의 범위 (i 이상 j 이하)를 구할 수 있다. 좌표의 최솟값은 X_i , 최댓값은 X_j , 중간값은 $X_{\lfloor (i+j)/2 \rfloor}$ 이다.

범위 내의 좌표 합은 좌표의 합의 누적합 배열 $S_i = X_1 + X_2 + \cdots + X_i = S_{i-1} + X_i$ 를 사전에 계산해 놓으면, $S_j - S_{i-1}$ 로 상수 시간에 계산할 수 있다. 질의마다 (i, j) 범위를 계산하는 데에 $O(\log N)$ 의 시간이 필요하다.

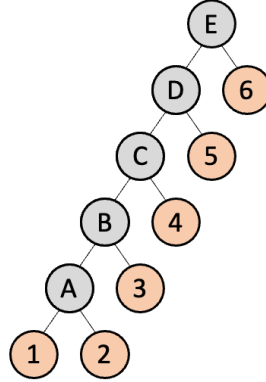
위와 같은 방식으로, 전체 시간복잡도 $O(N + Q \log N)$ 에 문제를 해결할 수 있다.

“중등부 3번 / 고등부 2번. 이진 트리” 문제 풀이

작성자: 박찬솔

부분문제 3

T_6 의 형태는 아래와 같다.



노드 A, B, C, D, E가 덮는 리프 노드의 구간은 $[1, 2]$, $[1, 3]$, $[1, 4]$, $[1, 5]$, $[1, 6]$ 이다. 따라서, $f(1, i) = 1$ 이다.

$i \geq 2$ 인 경우, 번호가 i 이상 j 이하인 리프 노드를 덮으려면, 번호가 i 이상 j 이하인 리프 노드들을 모두 선택할 수밖에 없다. 따라서, $f(i, j) = j - i + 1$ 이다.

즉, $S(T_N) = \sum_{j=1}^N 1 + \sum_{i=2}^N \sum_{j=i}^N (j - i + 1)$ 이다. 이 값은 $O(1)$ 에 계산할 수 있다.

부분문제 4

모든 (a, b) 순서쌍에 대해 $f(a, b)$ 의 값을 직접 구하려고 한다.

주어진 트리를 깊이 우선 탐색 등의 방법으로 순회하면, 각 노드 u 를 선택했을 때 덮이는 리프 노드 번호의 구간 $[l_u, r_u]$ 를 계산해 둘 수 있다.

$g(u, a, b)$ 를, “구간 $[a, b]$ 가 노드 u 의 서브 트리에 포함됨을 알고 있을 때, $f(a, b)$ 의 값”이라고 정의하자. $f(a, b)$ 의 값은 $g(\text{root}, a, b)$ 함수의 값을 계산하면 구할 수 있다. g 의 값을 재귀적으로 계산하여, $f(a, b)$ 의 값을 구할 것이다.

- $a = l_u$ 이고 $b = r_u$ 라면, 노드 u 를 선택하면 구간 $[a, b]$ 가 덮이므로, $g(u, a, b) = 1$ 이다. 노드 u 가 리프 노드였다면, 반드시 여기에 해당한다는 점에 유의하라.
- 노드 u 의 왼쪽 자식을 v , 오른쪽 자식을 w 라고 하자. $l_u = l_v$, $r_v + 1 = l_w$, $r_w = r_u$ 이다.
- 구간 $[a, b]$ 가 왼쪽 자식 v 의 서브 트리에 포함된다면 (즉 $b \leq r_v$ 라면), $g(u, a, b) = g(v, a, b)$ 이다.
- 구간 $[a, b]$ 가 오른쪽 자식 w 의 서브 트리에 포함된다면 (즉 $l_w \leq a$ 라면), $g(u, a, b) = g(w, a, b)$ 이다.
- 이제 $a \leq r_v < l_w \leq b$ 이다. 구간 $[a, r_v]$ 는 왼쪽 자식 v 의 서브 트리에 포함되고, 구간 $[l_w, b]$ 는 오른쪽 자식 w 의 서브 트리에 포함되며, 서로 겹치지 않으므로 독립적으로 계산할 수 있다. 즉, $g(u, a, b) = g(v, a, r_v) + g(w, l_w, b)$ 이다.

부분문제 1

위와 같은 g 의 정의는, 완전 이진 트리 형태로 표현된 세그먼트 트리에서, 닫힌 구간 $[a, b]$ 의 정보를 추출하기 위해 세그먼트 트리를 순회하는 함수와 그 형태가 유사하다. $f(a, b)$ 의 값은 세그먼트 트리에서 구간 $[a, b]$ 에 대한 정보를 계산하기 위해, 값을 가져와야 하는 노드의 최소 개수라고 생각할 수 있다.

해당 함수를 사용하여 $O(\log |T_N|)$ 시간에 $f(a, b)$ 를 구할 수 있다. 모든 (a, b) 순서쌍에 대해 $f(a, b)$ 의 값을 구한 후 합쳐주면 된다.

부분문제 5

부분문제 4의 풀이는 모든 순서쌍 (a, b) 에 대해, 구간 $[a, b]$ 를 덮기 위해 최소한으로 선택해야 하는 노드들을 직접 구한다고 볼 수 있다.

관점을 바꿔서, 각각의 노드에 대해, 위의 절차에서 그 노드를 선택하는 순서쌍 (a, b) 의 개수가 몇 개인지를 세어보자.

부분 문제 4에서와 같이, 트리 전체를 순회하여, 각 노드 u 를 선택했을 때 덮이는 리프 노드 번호의 구간 $[l_u, r_u]$ 을 계산해 두었다고 하자.

노드 u 의 부모 노드가 p 일 때, 구간 $[a, b]$ 를 덮을 때 노드 u 가 선택되는 조건은, 아래 두 조건과 같다.

- 노드 u 를 선택했을 때 구간 $[a, b]$ 바깥의 리프 노드가 덮이면 안 된다. 즉, $a \leq l_u \leq r_u \leq b$ 여야 한다.
- 노드 p 를 선택했을 때, 반드시 구간 $[a, b]$ 바깥의 리프 노드 하나 이상은 덮여야 한다. 만약 그렇지 않다면, u 대신 p 를 선택하면 되기 때문이다. 즉, $l_p < a$ 이거나 $b < r_v$ 여야 한다.

위의 두 조건을 동시에 만족하는 (a, b) 의 범위는 직사각형 형태로 나타나며, 그 개수를 $O(1)$ 시간에 구할 수 있다.

모든 노드 u 에 대해 위와 같이 (a, b) 의 개수를 세어 준 후 합쳐 주면, $S(T)$ 를 $O(T$ 의 노드 개수) 시간에 계산할 수 있다.

부분문제 6

부분문제 2와 같이 완전 이진 트리가 주어지는 경우 리프 노드 개수가 2^N 개로 상당히 많으므로, 모든 노드를 순회하는 방식으로 계산할 수 없다.

T 의 루트 노드가 u , u 의 왼쪽 자식 노드가 v , 오른쪽 자식 노드가 w 라고 하자. $f(a, b)$ 의 값을 제시된 대로 부분문제 4에서 제시된 것과 같이 여러 경우를 나누어서 계산해 보자.

$$\begin{aligned} S(T_u) &= \sum_{l_u \leq a \leq b \leq r_u} f(a, b) \\ &= \sum_{l_v \leq a \leq b \leq r_v} f(a, b) + \sum_{l_w \leq a \leq b \leq r_w} f(a, b) + \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} \{f(a, r_v) + f(l_w, b)\} - 1 \end{aligned}$$

마지막에 1을 빼 주는 이유는, $a = l_u, b = r_u$ 일 때 $f(a, b) = 1$ 인 케이스를 간단하게 처리해 주기 위함이다.

$\sum_{l_v \leq a \leq b \leq r_v} f(a, b)$ 는 v 의 서브 트리 내의 모든 (a, b) 순서쌍에 대한 $f(a, b)$ 의 합이므로, $S(T_v)$ 와 같다.

$\sum_{l_w \leq a \leq b \leq r_w} f(a, b)$ 는 w 의 서브 트리 내의 모든 (a, b) 순서쌍에 대한 $f(a, b)$ 의 합이므로, $S(T_w)$ 와 같다.

이제 $\sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} \{f(a, r_v) + f(l_w, b)\}$ 의 값을 계산하자. $f(a, r_v)$ 에는 b 가 나타나지 않고, $f(l_w, b)$ 에는 a 가 나타나지 않는다는 점에서 착안하여, 이 식을 전개하면 아래와 같다.

$$\begin{aligned} & \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} \{f(a, r_v) + f(l_w, b)\} \\ &= \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} f(a, r_v) + \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} f(l_w, b) \\ &= (r_w - l_w + 1) \sum_{l_v \leq a \leq r_v} f(a, r_v) + (r_v - l_v + 1) \sum_{l_w \leq b \leq r_w} f(l_w, b) \end{aligned}$$

$r_w - l_w + 1$ 는 w 의 서브 트리 내의 리프 노드 개수이다. $r_v - l_v + 1$ 는 v 의 서브 트리 내의 리프 노드 개수이다. 노드 u 의 서브 트리 내의 리프 노드 개수를 $\text{cnt}[u]$ 라고 정의하자.

$\sum_{l_v \leq a \leq r_v} f(a, r_v)$ 는 v 의 서브 트리에 있는 모든 리프 노드 a 에 대해, $f(a, r_v)$ 의 합이다. 이 값을 $\text{suffix}[v]$ 라고 정의하자.

$\sum_{l_w \leq b \leq r_w} f(l_w, b)$ 는 w 의 서브 트리에 있는 모든 리프 노드 b 에 대해, $f(l_w, b)$ 의 합이다. 이 값을 $\text{prefix}[w]$ 라고 정의하자.

이 값들을 관리할 수 있다면, $S(T_i) = S(T_{A_i}) + S(T_{B_i}) + \text{suffix}[A_i] \times \text{cnt}[B_i] + \text{prefix}[B_i] \times \text{cnt}[A_i] - 1$ 으로 계산할 수 있다.

$\text{cnt}[0] = 1$ 이고, $\text{cnt}[i] = \text{cnt}[A_i] + \text{cnt}[B_i]$ 이다.

$\text{prefix}[i] = \text{prefix}[A_i] + (\text{prefix}[B_i] + \text{cnt}[B_i] - 1)$ 이다. 왼쪽 서브 트리의 f 값들은 그대로 유지되고, 오른쪽 서브 트리의 f 값들은 1 증가하며, 루트 노드의 f 값은 1이기 때문이다.

$\text{suffix}[i] = (\text{suffix}[A_i] + \text{cnt}[A_i] - 1) + \text{suffix}[B_i]$ 이다. 오른쪽 서브 트리의 f 값들은 그대로 유지되고, 왼쪽 서브 트리의 f 값들은 1 증가하며, 루트 노드의 f 값은 1이기 때문이다.

따라서 필요한 모든 값들을 상수 시간에 계산할 수 있으며, 문제를 $O(N)$ 에 해결할 수 있다.